

System Architecture Document
Motion Detection Application
Version 1

1. System Overview

The Motion Detection Application is a local, standalone computer-vision application that captures a live feed from the system camera, detects movement within the camera frame, and displays a motion status directly on the live feed.

Version 1 is intentionally limited to motion detection. It does not identify, classify, or label objects.

The application operates entirely on the local machine and does not communicate with external services.

2. System Objectives

The Version 1 application shall:

1. Open the system's default camera.
2. Capture a continuous live video feed.
3. Detect changes or movement within the camera feed.
4. Display "Motion Detected" on the live feed when movement is detected.
5. Display a non-motion status when no significant movement is detected.
6. Allow the user to exit the application through a keyboard command.
7. Release the camera and application resources during shutdown.

The application shall not:

1. Connect to the Internet or any external network.
2. Upload or transmit camera data.
3. Save video frames.
4. Save screenshots or camera captures.
5. Record video.
6. Perform object detection.
7. Identify or label people, animals, or other objects.
8. Access third-party or network cameras in Version 1.

9. Technology Requirements

The application shall use:

- Python
- OpenCV
- NumPy

- Python Standard Library

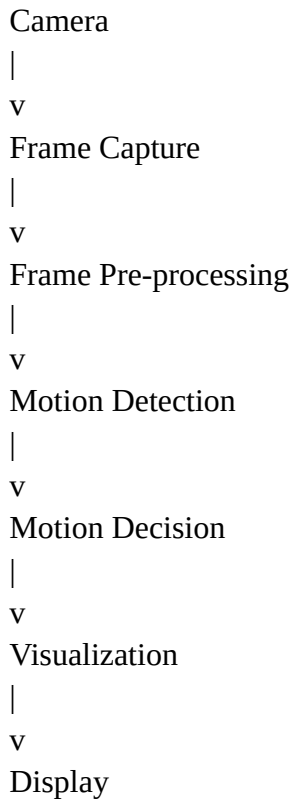
OpenCV shall provide the primary computer-vision and camera functionality.

NumPy shall support image/frame processing and numerical operations.

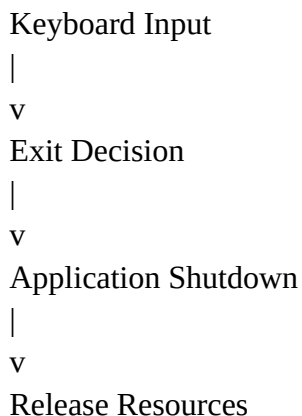
No additional third-party functional libraries shall be required for Version 1.

4. High-Level Architecture

The application shall use a modular processing pipeline:



A separate control path shall handle user input:



5. Major Components

5.1 Application Controller

Responsible for controlling the overall application lifecycle.

Responsibilities:

- Initialize configuration.
- Initialize the camera.
- Start the processing loop.
- Coordinate frame processing.
- Handle keyboard input.
- Handle application errors.
- Initiate shutdown.
- Ensure resources are released.

Suggested module:

main.py

5.2 Camera Manager

Responsible for interacting with the system camera through OpenCV.

Responsibilities:

- Open the default camera.
- Verify that the camera was successfully opened.
- Capture frames.
- Report frame capture failures.
- Release the camera during shutdown.

Suggested module:

camera.py

The camera interface should be designed so that future camera implementations can be added without significantly modifying the rest of the application.

Example interface:

```
open()
read()
is_open()
release()
```

5.3 Frame Pre-processing

Responsible for preparing frames for motion detection.

Possible operations include:

- Convert frame to grayscale.
- Apply light image smoothing.
- Prepare frames for comparison.

Pre-processing should remain independent from the motion-detection algorithm so that the algorithm can be changed in future versions.

Suggested module:

preprocessing.py

5.4 Motion Detector

Responsible exclusively for determining whether meaningful movement has occurred between frames.

A simple Version 1 implementation can use frame differencing:

1. Capture the current frame.
2. Convert it to grayscale.
3. Compare it with the previous processed frame.
4. Calculate the difference.
5. Apply a threshold.
6. Determine whether the amount of change exceeds a configured threshold.
7. Return a motion/no-motion result.

The detector must not attempt to determine what caused the movement.

Suggested module:

motion_detector.py

Example result:

MotionResult:

motion_detected: Boolean

motion_area: Integer

confidence: Not required in Version 1

The system should avoid treating minor camera noise or insignificant pixel changes as meaningful motion by using configurable thresholds.

5.5 Renderer

Responsible for displaying application information on the camera feed.

Responsibilities:

- Display the live camera frame.
- Display "Motion Detected" when motion is detected.
- Display "No Motion" when no significant motion is detected.
- Display optional diagnostic information when debug mode is enabled.

Suggested module:

renderer.py

The renderer should not contain motion-detection logic.

5.6 Configuration

Centralizes configurable application parameters.

Suggested module:

config.py

Potential configuration values:

- Camera index.
- Motion threshold.
- Minimum motion area.
- Frame processing resolution.
- Exit key.
- Debug/logging level.
- Display window name.

Keeping these values outside the processing logic allows future versions to modify behavior without restructuring the application.

5.7 Logging and Debugging

The application shall provide a structured logging mechanism using the Python Standard Library logging module.

Suggested logging levels:

ERROR

Critical application failures.

WARNING

Recoverable problems, such as temporary frame-read failures.

INFO

Major application events, such as camera initialization and shutdown.

DEBUG

Detailed processing information useful during development and testing.

Example debug information:

- Camera opened.
- Camera failed to open.
- Frame captured.
- Frame capture failure.
- Motion detected.
- Motion not detected.

- Motion measurement.
- Application shutdown.
- Resource release.

Logging should not store camera frames or image data.

Logs should contain application state and diagnostic information only.

8. Motion Detection Pipeline

The normal processing loop shall operate approximately as follows:

1. Open the system camera.
2. Verify that the camera is available.
3. Capture a frame.
4. Validate the frame.
5. Pre-process the frame.
6. Compare the current frame with the previous frame.
7. Calculate the amount of visual change.
8. Compare the change against configured thresholds.
9. Produce a motion/no-motion result.
10. Render the result on the live feed.
11. Display the frame.
12. Check for the configured exit key.
13. Repeat until shutdown.
14. Error Handling

The application should handle common failures without crashing unnecessarily.

Camera cannot be opened:

- Log an ERROR.
- Display an appropriate error message.
- Terminate cleanly.

Frame cannot be read:

- Log a WARNING.
- Skip the invalid frame.
- Continue processing where possible.

Invalid or empty frame:

- Do not perform motion calculations.

- Do not replace a valid previous frame with invalid data.
- Log the condition at DEBUG or WARNING level.

Display failure:

- Log the failure.
- Initiate controlled shutdown.

Unexpected exception:

- Log the exception and stack trace.
- Release camera resources.
- Destroy OpenCV windows.
- Terminate safely.

Resource cleanup should occur through a finally-style shutdown path so that the camera is released even when an unexpected error occurs.

10. Testing and Quality Assurance

Testing shall be performed at multiple levels.

10.1 Unit Testing

The motion-detection logic should be designed so that core calculations can be tested without requiring a physical camera.

Tests should include:

- Identical frames produce no motion.
- Frames with significant changes produce motion.
- Small pixel changes do not incorrectly trigger motion.
- Configured thresholds are respected.
- Empty frames are rejected.
- Invalid frame data is handled safely.
- Initial frame handling works correctly.

10.2 Integration Testing

Integration tests should verify:

- Camera initialization.
- Frame capture.
- Pre-processing.
- Motion detection.
- Rendering.

- Keyboard exit.
- Resource cleanup.

10.3 End-to-End Testing

A complete test should verify:

1. Application starts.
2. Camera feed appears.
3. Stationary scene produces "No Motion".
4. User movement produces "Motion Detected".
5. Application continues processing after movement stops.
6. Exit key terminates the application.
7. Camera and display resources are released.

10.4 Fault Testing

The application should also be tested against:

- Camera unavailable.
- Camera disconnected during operation.
- Invalid frames.
- Very dark or noisy scenes.
- Rapid movement.
- Small movements.
- Continuous movement.
- Repeated exit-key presses.

10.5 Performance Testing

Version 1 should target real-time operation.

A reasonable initial target is:

- Processing resolution: approximately 640x480.
- Target frame rate: 30 FPS or greater where supported by the hardware.
- No unnecessary frame storage.
- No unbounded memory growth.
- Processing should not continuously consume excessive CPU resources.

Performance measurements can include:

- FPS.
- Frame-processing time.

- CPU usage.
- Memory usage.
- Number of failed frame reads.

11.Privacy and Data Handling

The application is designed as a local-only application.

Camera frames exist only in memory while being processed.

The application shall not:

- Write frames to disk.
- Write screenshots to disk.
- Record video.
- Upload frames.
- Send frames to another machine.
- Connect to cloud services.
- Connect to external APIs.

Debug logs must not contain raw image data.

12.Scalability and Future Architecture

Although Version 1 is intentionally small, the architecture shall separate camera access, image processing, motion detection, rendering, configuration, and application control.

This separation allows future functionality to be added without replacing the entire application.

Possible future versions may introduce:

Version 2:

- Improved motion detection.
- Motion regions.
- Movement statistics.
- More sophisticated filtering.

Version 3:

- Multiple detection modes.
- Advanced visualization.
- Configurable detection profiles.

Version 4:

- Additional camera management capabilities.

Version 5:

- Third-party or network camera support.

Third-party camera support is explicitly outside the Version 1 architecture, but the Camera Manager interface should prevent future camera implementations from being tightly coupled to the motion-detection system.

The motion detector should receive processed image data rather than directly accessing the camera. This allows future camera sources to use the same detection pipeline.

13. Proposed File Structure

```

motion_detection_app/
|
+-- main.py
+-- camera.py
+-- preprocessing.py
+-- motion_detector.py
+-- renderer.py
+-- config.py
+-- logger.py
|
+-- tests/
+-- test_motion_detector.py
+-- test_preprocessing.py
+-- test_camera.py

```

The structure is intentionally modular while remaining small enough for a Version 1 implementation.

14. Architectural Principles

The Version 1 implementation should follow these principles:

- Single responsibility: each module should have one primary responsibility.
- Low coupling: motion detection should not depend directly on camera hardware.
- Configurability: thresholds and operational settings should be centralized.
- Testability: core processing should be testable without physical hardware.
- Local processing: all camera processing remains on the local machine.
- No unnecessary data persistence: frames exist only as required for processing.
- Extensibility: future camera sources and processing capabilities should be addable without rewriting the application.
- Controlled shutdown: all hardware and OpenCV resources must be released.

15. Version 1 Definition of Done

Version 1 is considered complete when:

1. The application successfully opens the system camera.

2. A live camera feed is displayed.
3. Motion within the feed can be detected.
4. "Motion Detected" is displayed when meaningful motion occurs.
5. A non-motion state is displayed when no meaningful motion occurs.
6. The application does not perform object detection or object labeling.
7. No frames or captures are saved.
8. No network connection is required.
9. Camera and display resources are released correctly.
10. Core motion-detection behavior has automated tests.
11. Application failures generate useful diagnostic logs.
12. The architecture permits future camera and computer-vision functionality to be added without major restructuring.